A Artifact Appendix

A.1 Abstract

This artifact contains the complete source code of NIXT (NCCL Inspector eXporter Tool), the observability pipeline used in the paper: (1) the **Inspector** NCCL profiler plugin (C++), which attaches to any NCCL-based job and emits per-rank, per-collective JSON dumps; (2) the **exporter** (Python), which parses the dumps into parquet tables and summary reports; and (3) the **analysis** scripts (Python) that generated every experiment figure in the paper.

The telemetry analyzed in the paper was collected from large-scale production LLM training runs (up to 2048 H100 GPUs) whose data is confidential and cannot be redistributed. The artifact is therefore *code only*: we claim the **Available** and **Reviewed** badges, not **Reproducible**. To let evaluators verify functionality end-to-end, the artifact includes a single-script demo that runs the full pipeline (plugin → dumps → parquet → summaries) on a small GPU node using `nccl-tests` as the workload, plus reference copies of all paper figures in `expected_output/` and a figure-to-script map in `docs/figure_map.md`.

A.2 Artifact check-list (meta-information)

- **Program:** NCCL Inspector profiler plugin (C++); exporter and analysis scripts (Python); `nccl-tests` (fetched automatically by the demo).
- **Compilation:** GNU make, `g++` (C++14/17), CUDA toolkit ≥ 12.x.
- **Binary:** `libnccl-profiler-inspector.so`, built from source.
- **Data set:** none distributed (production telemetry is confidential); the demo generates its own data via `nccl-tests`.
- **Run-time environment:** Linux x86_64; CUDA ≥ 12.x; NCCL ≥ 2.28 (2.28–2.30 tested); Python ≥ 3.10.
- **Hardware:** one node with ≥ 2 NVIDIA GPUs (developed on A100/H100; no architecture-specific code).
- **Execution:** single script `demo/run_demo.sh`.
- **Metrics:** per-collective bus bandwidth, execution time, message sizes, and transferred bytes.
- **Output:** per-rank `*.log.gz` JSON dumps; parquet tables; summary CSVs and plots; reference paper figures in `expected_output/`.
- **Experiments:** end-to-end functional demo; paper analysis scripts provided for inspection and reuse.
- **How much disk space required (approximately)?:** ∼2 GB (dominated by the `nccl-tests` build).
- **How much time is needed to prepare workflow (approximately)?:** ∼15 minutes.
- **How much time is needed to complete experiments (approximately)?:** ∼15 minutes.
- **Publicly available?:** Yes (GitHub; archived on Zenodo).
- **Code licenses (if publicly available)?:** BSD-3-Clause (Inspector plugin vendored from NVIDIA's NCCL `ext-profiler` example, NVIDIA copyright).
- **Archived (provide DOI)?:** `10.5281/zenodo.XXXXXXX` (TODO).

A.3 Description

A.3.1 How to access

The artifact is available at `https://github.com/ziyang-arch/nixt-analysis` and archived at `10.5281/zenodo.XXXXXXX`. Repository layout:

- `inspector/` — NCCL Inspector profiler plugin (C++, standalone Makefile).
- `exporter/` — `perf_summary_exporter.py`: parses dumps into parquet + summary reports/plots.
- `analysis/` — paper figure scripts.
- `demo/` — end-to-end functional demo (2–8 GPUs).
- `expected_output/` — experiment figures exactly as they appear in the paper.
- `docs/figure_map.md` — paper figure ↔ script ↔ input mapping.

A.3.2 Hardware dependencies

A Linux x86_64 node with at least 2 NVIDIA GPUs. Any recent architecture works; the demo was developed against A100/H100.

A.3.3 Software dependencies

CUDA ≥ 12.x; NCCL ≥ 2.28 (profiler plugin API; 2.28–2.30 tested); Python ≥ 3.10 with `pandas`, `tqdm`, `duckdb`, `matplotlib`, `pyarrow`, `numpy` (`pip install -r exporter/requirements.txt`); `nccl-tests` (cloned and built automatically by the demo).

A.3.4 Data sets

The paper's telemetry comes from confidential production training runs and is not redistributable. The demo generates its own Inspector dumps from `nccl-tests` collectives.

A.4 Installation

```
pip install -r exporter/requirements.txt
make -C inspector   # set CUDA_HOME if CUDA is
                    # not at /usr/local/cuda
```

A.5 Experiment workflow

The pipeline is: Inspector plugin (attached to a NCCL job via `NCCL_PROFILER_PLUGIN`) → per-rank `*.log.gz` JSON dumps → exporter → parquet + summaries → analysis scripts → figures. The demo automates all stages:

```
NGPUS=2 ./demo/run_demo.sh
```

It (1) builds the plugin, (2) clones/builds `nccl-tests` if needed, (3) runs `all_reduce_perf` and `all_gather_perf` (1 KB–256 MB) with the plugin attached, and (4) exports the dumps to parquet and summary reports.

A.6 Evaluation and expected results

After the demo completes (∼15 min total), verify:

- `demo/dump/<timestamp>/` contains per-rank `*.log.gz` Inspector dumps (the script fails loudly if none are produced);
- `data/demo-analysis/parquet_files/` contains one parquet file per rank;
- `data/demo-analysis/` contains summary CSVs and plots with plausible bus-bandwidth and message-size statistics for the two collectives.

This validates the full plugin → exporter path used for every result in the paper. The scripts in `analysis/` generated all experiment figures; `docs/figure_map.md` maps each paper figure to its script, and `expected_output/` contains the exact figures from the camera-ready paper. Because the underlying telemetry is confidential, re-generating those figures is out of scope for this evaluation (we do not claim the **Reproducible** badge).

A.7 Experiment customization

The plugin attaches to any NCCL/PyTorch job:

```
export NCCL_PROFILER_PLUGIN=\
  /path/to/libnccl-profiler-inspector.so
export NCCL_INSPECTOR_ENABLE=1
export NCCL_INSPECTOR_DUMP_DIR=/path/to/dumps
<launch your job as usual>
python3 exporter/perf_summary_exporter.py \
  --input_dir /path/to/dumps
```

All analysis scripts resolve inputs/outputs relative to `NIXT_ROOT` (defaults to the repository root), so they can be pointed at telemetry collected from the evaluator's own workloads. `inspector/README.md` documents all knobs (verbose event traces, Prometheus export, dump intervals).

A.8 Methodology

Submission, reviewing and badging methodology:

- `https://www.acm.org/publications/policies/artifact-review-and-badging-current`
- `https://cTuning.org/ae`